Université de Lorraine · IUT Nancy-Charlemagne · BUT 3 informatique

NRV.net
Billetterie du festival Nancy Rock Vibration

Dossier de rendu de l'itération 1 : conception, maquettes et première version du backend

SAE « Atelier-projet de développement web 1 »
Session du 19 au 23 octobre 2026

Livrable 1 · V1 opérationnelle
Version du 20 octobre 2026

Sommaire

Ce dossier présente le travail réalisé pendant l'itération 1 : l'analyse du sujet, les choix de conception, les maquettes des écrans, l'environnement technique et la première version de l'API.

1. **Contexte et objectifs**
   Le festival · Fonctionnalités attendues · Contraintes techniques
2. **Organisation du projet**
   Trello · Git et conventions · Outils
3. **Conception**
   Scénarios d'usage et cas d'utilisation · Règles métier · Diagramme de classes · Base de données · Contrat d'API
4. **Maquettes des écrans**
   Programme · Soirée · Panier et Pass 3 jours · Connexion et paiement · Billets · Backoffice
5. **Environnement technique**
   Docker Compose · Jeu de données de test · Installation
6. **Backend réalisé (itération 1)**
   Architecture hexagonale · Arborescence · Parcours d'une requête · Endpoints · Sécurité · Tests
7. **Bilan et perspectives**
   État d'avancement · Itération 2 · Déploiement

1 Contexte et objectifs

NRV.net est l'application de billetterie du festival **Nancy Rock Vibration**. Elle permet de consulter le programme, d'acheter des billets en ligne et, pour l'organisation, de suivre le remplissage des soirées.

Le festival

- Le festival dure **2 semaines**. Chaque **soirée** a lieu à une date unique, dans un lieu donné, avec une jauge (nombre maximal de places).
- Une soirée regroupe **2 à 3 spectacles**. Chaque spectacle a **un seul style** de musique (classic rock, blues rock, métal, chanson…), une soirée peut mélanger les styles.
- Un **billet** donne accès à tous les spectacles de sa soirée. Deux tarifs : **normal** et **réduit** (étudiants, demandeurs d'emploi, personnes en situation de handicap).
- Le **Pass 3 jours** est un billet spécifique qui donne accès à **3 soirées distinctes**, avec une remise de 3 %. Il existe aussi en normal et en réduit.
- L'achat se fait uniquement en ligne, après inscription, et seulement s'il reste des places. Le billet n'est pas nominatif : il est accessible par une URL contenant un jeton aléatoire.

Fonctionnalités attendues

Réf.	Fonctionnalité du sujet	État à l'itération 1
F1	Liste des spectacles : titre, date, horaire, image	**Réalisé (API)**
F2	Détail d'une soirée : nom, thématique, date, lieu, tarifs, spectacles (artistes, description, style, vidéo)	**Réalisé (API)**
F3	Clic sur un spectacle → détail de sa soirée	**Réalisé (lien HATEOAS)**
F4 · F5	Filtrage de la liste par date et par style	**Réalisé (API)**
F6 · F7	Ajout au panier, état du panier et total	Maquetté
F8 · F9	Validation en commande, paiement simulé avec contrôle et mise à jour des places	Conçu
F10	Billets imprimables (nom de l'acheteur, référence, date, catégorie)	Maquetté, CSS d'impression prêt
F11	Backoffice : places vendues et recette par soirée	Vue SQL et maquette
E1	Pagination de la liste des spectacles	**Réalisé (API)**
E2 · E4	Modification du panier, vérification des places à l'ajout	Maquetté
E3	Backoffice : ajout de spectacles et de soirées	Non commencé

Contraintes techniques

Backend

- PHP 8 / Slim 4 dans des conteneurs Docker
- API RESTful en JSON, liens HATEOAS
- Architecture hexagonale stricte, injection de dépendances (PHP-DI)
- PDO avec requêtes préparées, validation des données
- PSR-1 et PSR-4, tests unitaires (PHPUnit)

Frontend

- HTML5 sémantique, valide W3C
- Sass/SCSS, responsive (mobile, tablette, ordinateur)
- CSS Grid et Flexbox, feuille `@media print` pour le billet
- Données chargées par `fetch` et affichées avec Handlebars

L'application doit être déployée dans des conteneurs Docker sur **docketu.iutnc.univ-lorraine.fr**.

2 Organisation du projet

Trello

Le travail est planifié par étapes sur un tableau Trello ; chaque carte correspond à un ou plusieurs commits.

Étape	Cartes
1. Conception et spécifications	scénario d'usage · diagramme de classes · BDD et schéma SQL · wireframes de la grille, de la fiche soirée · maquettes du panier, du formulaire de commande, du billet · wireframe du backoffice · mise en commun
2. Environnement	dépôt GitHub · docker compose · script SQL · script d'injection des données · structure Sass/SCSS · squelette de l'architecture hexagonale
3. Production	[BACK] arborescence et conteneur de dépendances · [BACK] entités et interfaces repository · [BACK] adaptateurs PDO · [BACK] actions Slim et routes · [FRONT] socle Sass et layouts · [FRONT] templates Handlebars et fetch · [FRONT] filtrage dynamique · [FRONT] vue détaillée d'une soirée

Git

- Dépôt du groupe : **Atelier-Projet-de-developpement-web-1**, branche de travail `production-kam`.
- Un commit par carte, dont le message reprend le nom de la carte : `2.1 docker-compose et config`, `[back] adaptateurs pdo`…
- Les commentaires du code sont courts et rédigés en français.

Historique de l'itération 1

```
2.1 docker-compose et config
2.2 seed fixtures
2.3 arborescence hexagonale
1.3 schema bdd pass 3 jours en billet
2.2 seed fixtures avec pass 3 jours
initialiser la structure sass/scss
fix volumes docker pour selinux (fedora)
fix composer : php-jwt 7 (faille de securite)
[back] entites domaine et interfaces repository
[back] adaptateurs pdo
2.2 seed : groupes en plus pour eviter les noms
de secours
[back] actions slim et routes api
[back] arborescence hexagonale et conteneur de
dependances
[back] ports de sortie dans application/port/out
tests unitaires phpunit du domaine et des cas
d'usage
preparer le deploiement docketu (ports, cors,
url de l'api)
readme : installation, donnees de test, api,
tests, deploiement
1.5 maquettes visuelles des ecrans
```

La documentation accompagne le code dans le dépôt : `README.md` (installation, données de test, API, tests, déploiement), `docs/arborescence.md`, `docs/maquettes/`, `docs/uml/` (sources des diagrammes) et `sql/01_schema.sql`.

3 Conception

3.1 Scénarios d'usage

Trois acteurs utilisent l'application : le **visiteur** (anonyme, il consulte), l'**acheteur** (inscrit, il commande et paie) et l'**administrateur** (il suit les jauges).

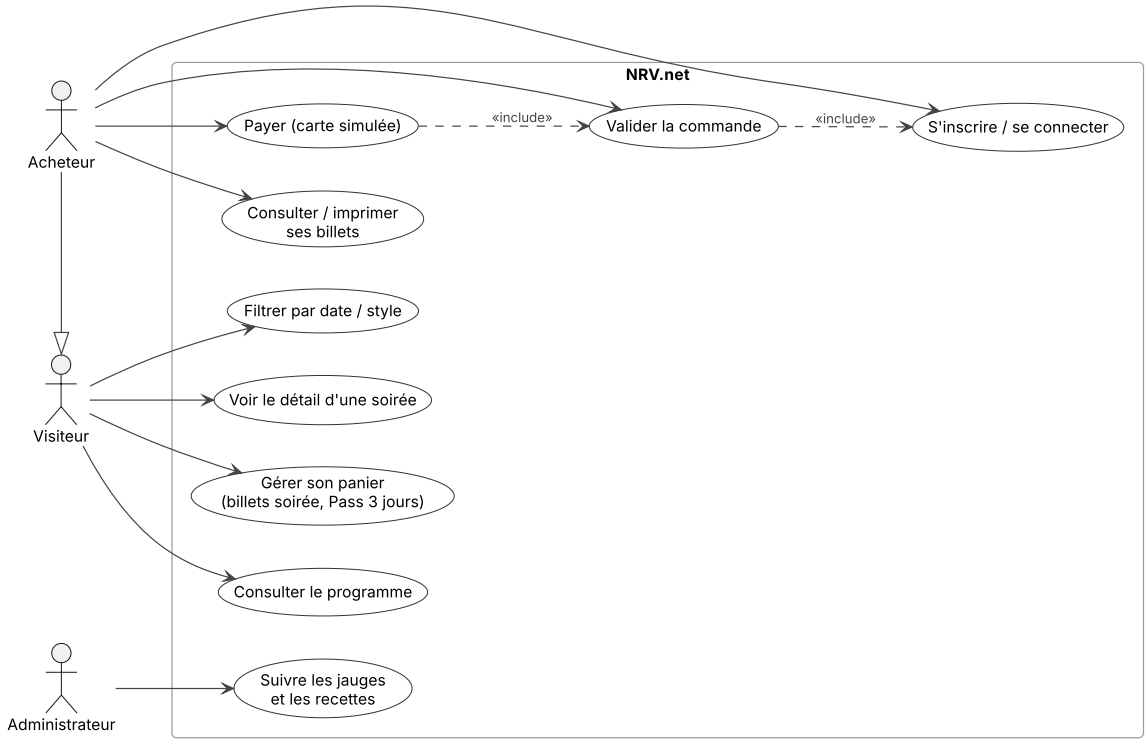


Figure 1 · Cas d'utilisation de NRV.net

Parcours A · Léa découvre le programme (visiteur, sur téléphone)

1. Elle ouvre l'accueil : la grille des spectacles s'affiche (image, titre, date, horaire), 12 par page.
2. Elle choisit le style « Métal » puis la date du 11 juin : la grille se met à jour sans recharger la page.
3. Elle touche une carte : le détail de la soirée s'ouvre (lieu, tarifs, places restantes, les 3 spectacles avec leur vidéo).

Parcours B · Marc achète des billets (acheteur, sur ordinateur)

1. Sur « Nuit métal », il ajoute 2 billets au tarif normal. Le site vérifie qu'il reste assez de places.
2. Dans le panier, il compose un **Pass 3 jours** réduit pour sa fille (11, 12 et 15 juin) : 58,20 € au lieu de 60 €.
3. Il valide le panier, se connecte (ou s'inscrit) : la commande est créée au statut « en attente ».
4. Il paie avec une carte (simulée). Le serveur vérifie les places **au moment du paiement**, les réserve et crée les billets, chacun avec un jeton unique.
5. Il imprime ses billets : une page A4 sobre, sans menu, grâce à la feuille `@media print`.

Cas d'erreur prévus : carte invalide (422), soirée devenue complète entre-temps (409, rien n'est réservé), double clic sur « Payer » (409, pas de billet en double), session expirée (401).

Parcours C · Sophie suit les ventes (administratrice)

Elle ouvre le backoffice : pour chaque soirée, la jauge, les places vendues (billets et pass), les places restantes, le taux de remplissage et la recette. Les soirées complètes ou presque complètes sont mises en évidence.

3.2 Règles métier

Règle	Comment elle est appliquée
Une soirée a 2 à 3 spectacles	l'entité `Soiree` refuse les autres cas (`NombreSpectaclesInvalideException`)
Un spectacle a un seul style	clé étrangère `spectacles.style_id` ; l'entité `Spectacle` reçoit un seul `Style`
Tarifs et jauge portés par la soirée	colonnes `tarif_normal`, `tarif_reduit`, `jauge`, `places_vendues` ; tarif réduit ≤ normal ; jauge ≤ capacité du lieu
On ne vend pas au-delà de la jauge	`Soiree::reserver()` lève `SoireeCompleteException` ou `PlacesInsuffisantesException` ; contrainte `CHECK` en base en plus
Pass 3 jours = un billet spécifique	formule `pass` en base (87,30 € / 58,20 €) ; billet de type `PASS` relié à 3 soirées distinctes ; il prend une place dans chacune
Places contrôlées au paiement	transaction : verrouillage des soirées, contrôle, réservation, création des billets ; tout est annulé au premier refus
Billet non nominatif, accessible par URL	jeton UUID v4 aléatoire unique ; le nom de l'acheteur est seulement imprimé
Montants exacts	tous les prix sont stockés en centimes (entiers), pas de flottants

3.3 Diagramme de classes

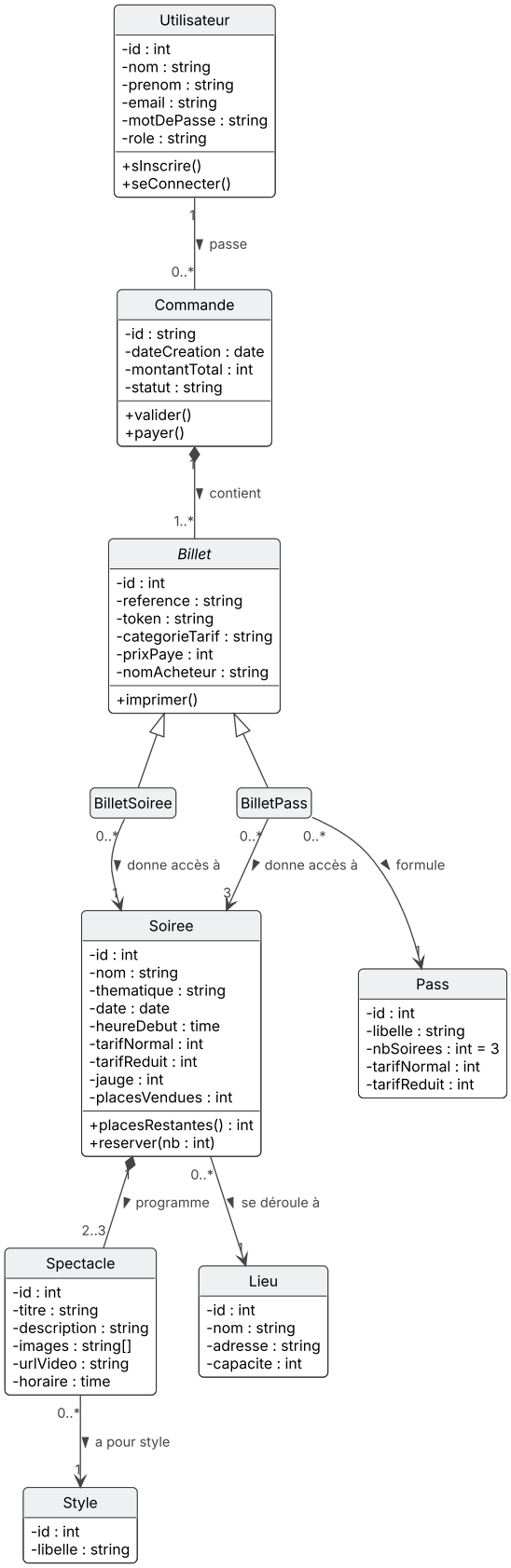


Figure 2 · Modèle du domaine

Comment le lire

- **Utilisateur → Commande → Billet** : un utilisateur passe des commandes ; une commande contient ses billets (composition, losange plein).
- **Billet** est abstrait, avec deux sortes :
  - **BilletSoiree** donne accès à **1** soirée ;
  - **BilletPass** donne accès à **3** soirées et suit une formule **Pass** (tarif normal ou réduit).
- **Soiree** porte les tarifs, la jauge et les places vendues ; elle *programme* 2 à 3 **Spectacle** (composition) et *se déroule* dans un **Lieu**.
- Chaque **Spectacle** a exactement **un Style**.

> **Correspondance en base**
>
> En base, l'héritage `Billet` est représenté par une seule table `billets` avec une colonne `type` (SOIREE ou PASS). Les 3 soirées d'un pass sont dans la table `billet_pass_soirees`.

> **Choix de modélisation**
>
> - le Pass 3 jours est un billet à part entière, et non une remise calculée sur le panier ;
> - le style est une classe, reliée à chaque spectacle par une multiplicité 1 ;
> - le panier n'est pas une classe : il vit dans le navigateur et n'est pas enregistré ;
> - le paiement simulé est une méthode de `Commande`.

3.4 Base de données

MariaDB 10.11, 14 tables InnoDB en `utf8mb4`. Le script complet est `sql/01_schema.sql` ; il est joué automatiquement au premier démarrage du conteneur.

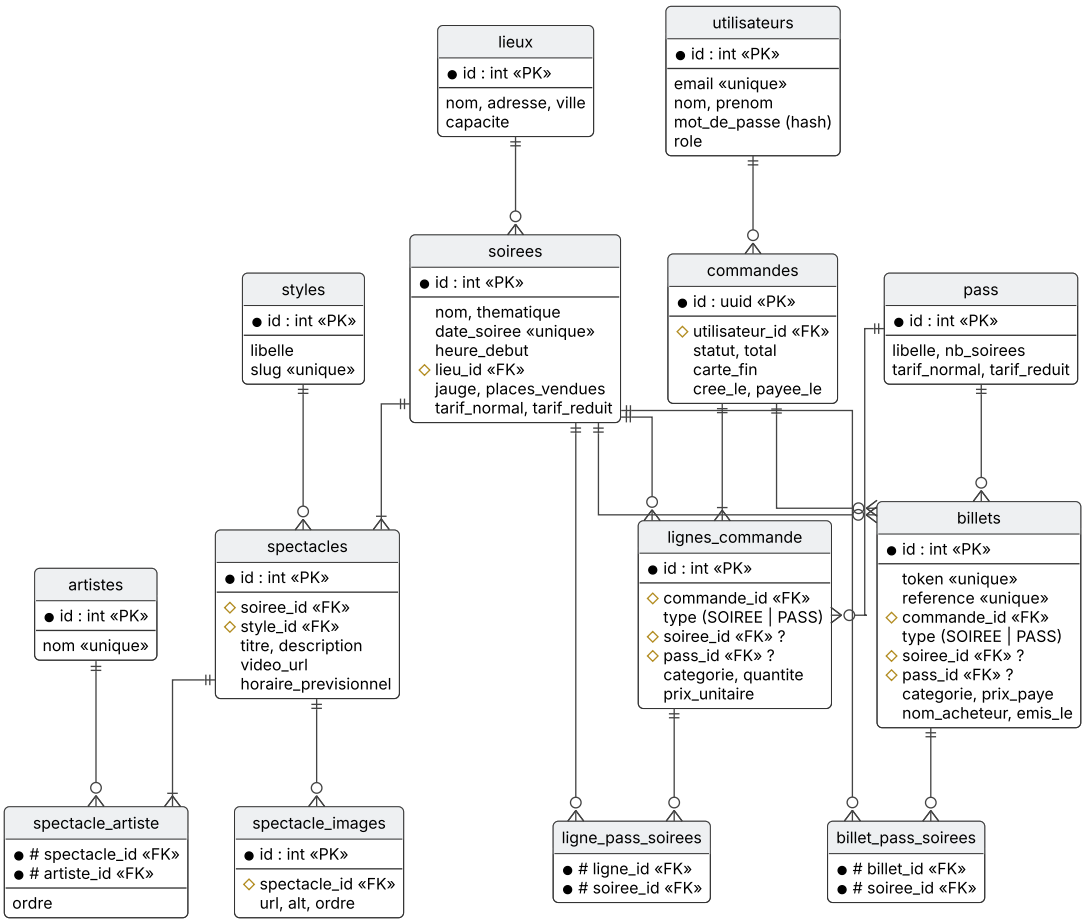


Figure 3 · Schéma relationnel (PK clé primaire, FK clé étrangère, ? = facultatif)

Tables	Rôle
`lieux`, `styles`, `artistes`	données de référence
`soirees`, `spectacles`	le programme ; `soirees.places_vendues` est le compteur de la jauge
`spectacle_artiste`, `spectacle_images`	un spectacle a un ou plusieurs artistes et une ou plusieurs images
`pass`	la formule Pass 3 jours et ses deux tarifs
`utilisateurs`, `commandes`, `lignes_commande`, `ligne_pass_soirees`	les comptes et les commandes ; une ligne = n billets identiques (soirée ou pass)
`billets`, `billet_pass_soirees`	les billets émis au paiement, avec leur jeton unique
vue `v_jauges_soirees`	pour le backoffice : jauge, vendues, restantes, taux, recette (un pass compte pour un tiers dans chacune de ses soirées)

Contraintes notables : une soirée par date (`UNIQUE`), `places_vendues ≤ jauge` , tarif réduit ≤ normal, cohérence type / soirée / pass sur les billets (`CHECK`), jeton et référence uniques.

3.5 Contrat d'API

Toutes les réponses sont en JSON. Une ressource : `{"type":"resource", "<nom>":{…}, "links":{…}}` ; une liste : `{"type":"collection", "count", "total", …, "links"}`. Les erreurs suivent le format RFC 7807 (`application/problem+json`).

Verbe	Route	Rôle	État
GET	`/spectacles`	liste paginée, filtres `date` et `style`	**Réalisé**
GET	`/spectacles/{id}`	un spectacle	**Réalisé**
GET	`/soirees/{id}`	soirée complète avec ses spectacles	**Réalisé**
GET	`/styles` , `/dates`	valeurs des filtres	**Réalisé**
GET	`/pass`	formule Pass 3 jours et ses tarifs	Itération 2
POST	`/auth/register` , `/auth/signin`	inscription, connexion (jeton JWT)	Itération 2
POST	`/commandes`	panier → commande « en attente »	Itération 2
POST	`/commandes/{id}/paiement`	paiement simulé, réservation des places, création des billets	Itération 2
GET	`/commandes` , `/commandes/{id}/billets`	mes commandes et mes billets	Itération 2
GET	`/billets/{token}`	un billet, par son jeton	Itération 2
GET	`/admin/jauges`	jauges et recettes (rôle admin)	Itération 2

Formats : montants en centimes (`3000` = 30,00 €), dates ISO (`2027-06-11`), heures `HH:MM` .

4 Maquettes des écrans

Les maquettes sont des pages HTML statiques (`docs/maquettes/`) réalisées avec la feuille de style Sass du projet et les données du jeu de test. Elles fixent la structure HTML, la mise en page responsive et les composants de chaque écran ; le front de l'itération 2 remplace leur contenu statique par les données de l'API.

4.1 Programme du festival

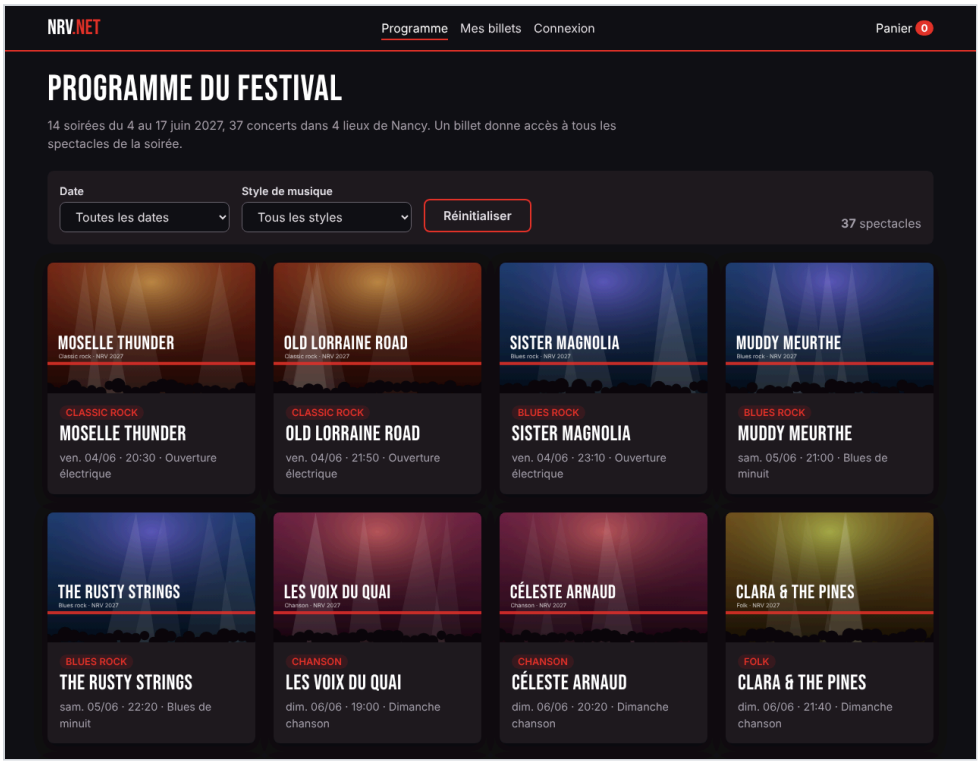


Figure 4 · Programme sur ordinateur : filtres, grille, pagination

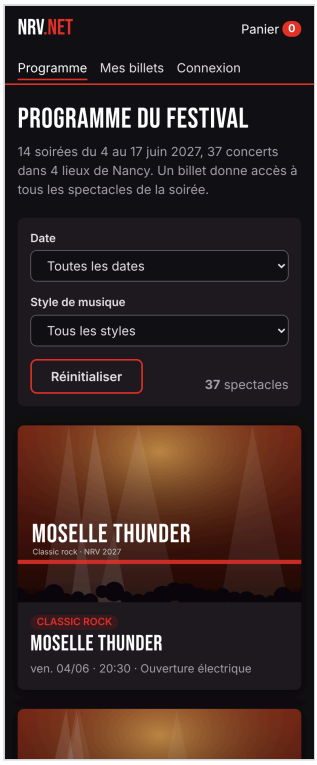


Figure 5 · Sur téléphone

Chaque carte est un lien vers la soirée (F3). Les filtres date et style se cumulent (F4, F5) et la grille se met à jour sans rechargement ; la pagination montre 12 spectacles par page (E1).

4.2 Détail d'une soirée

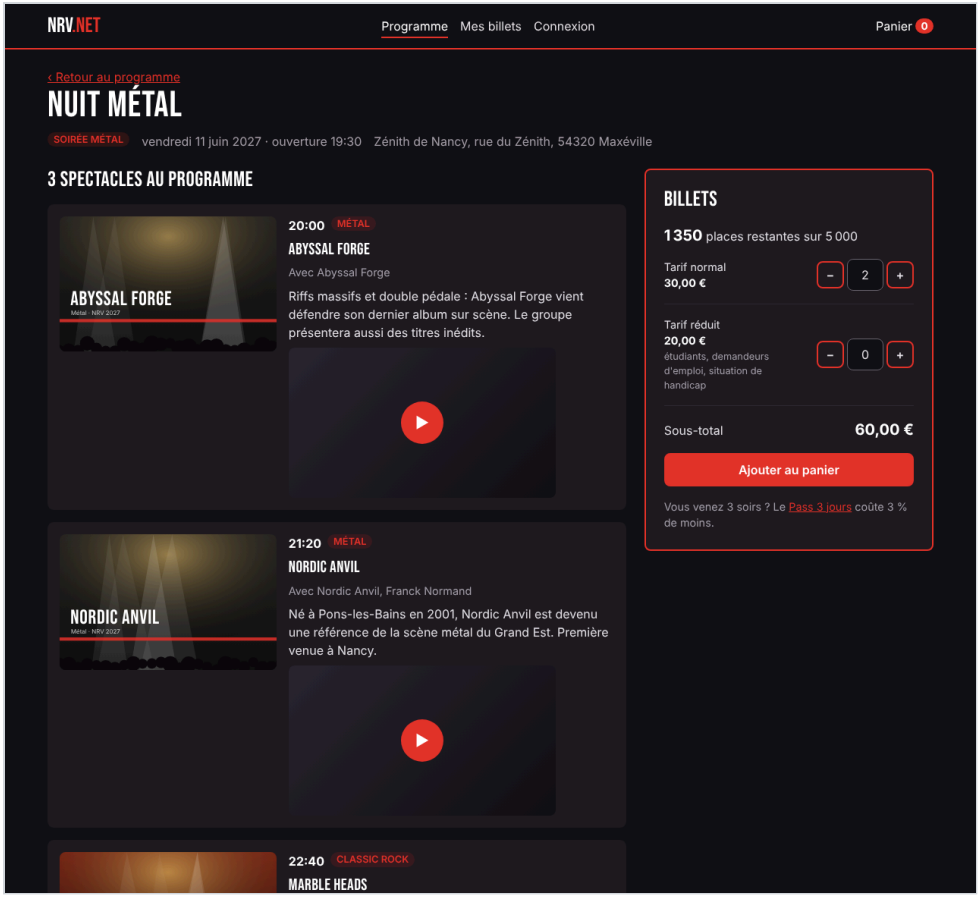


Figure 6 · Soirée « Nuit métal » : 3 spectacles et encart d'achat



Figure 7 · Sur téléphone

L'encart d'achat reste visible pendant le défilement sur ordinateur (`position: sticky`). Il affiche les places restantes, ce qui permet de bloquer un ajout impossible (E4).

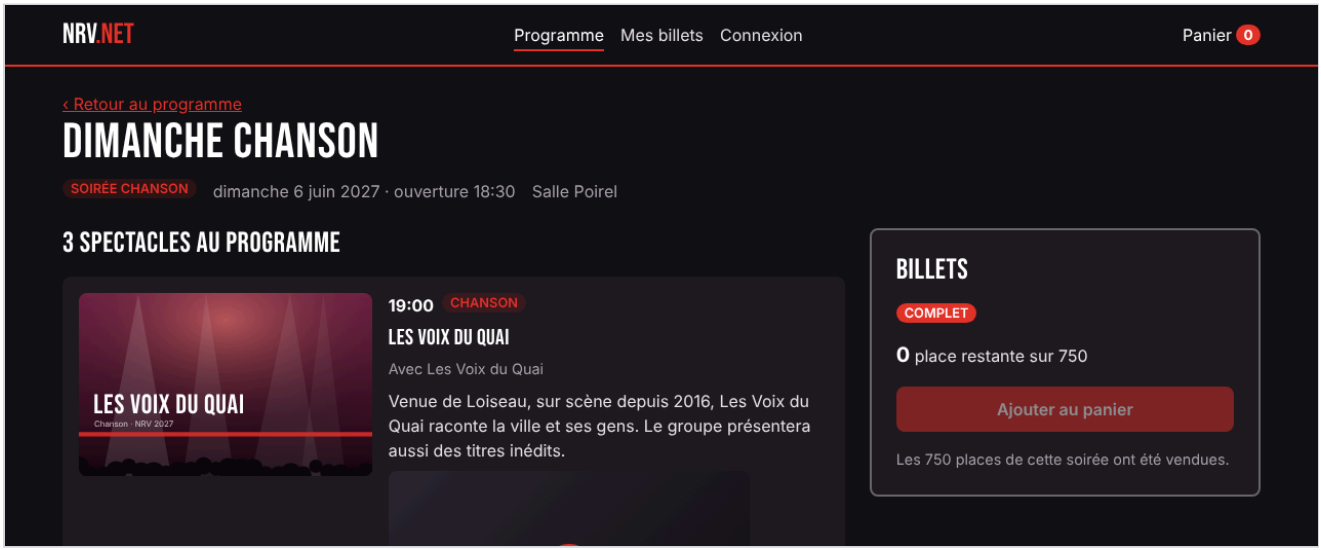


Figure 8 · Soirée complète : le bouton d'achat est désactivé

4.3 Panier et Pass 3 jours

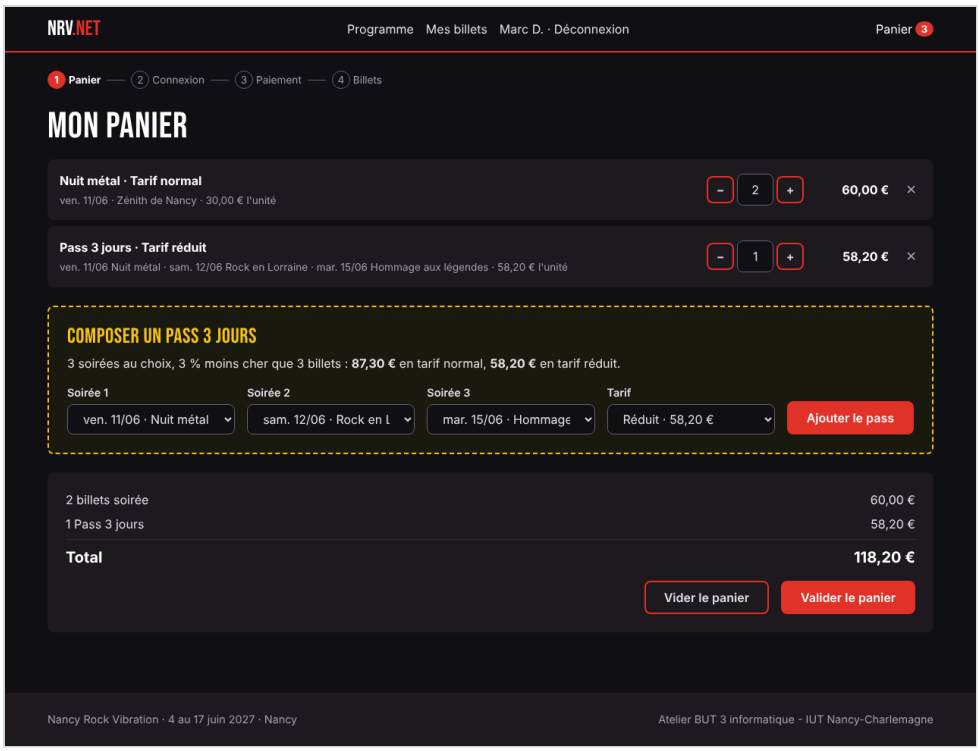


Figure 9 · Panier : billets soirée, Pass 3 jours, total



Figure 10 · Sur téléphone

Chaque ligne se modifie avec – et + (E2). L'encart « Composer un Pass 3 jours » propose 3 soirées et le tarif ; le pass devient une ligne du panier.

4.4 Connexion et paiement

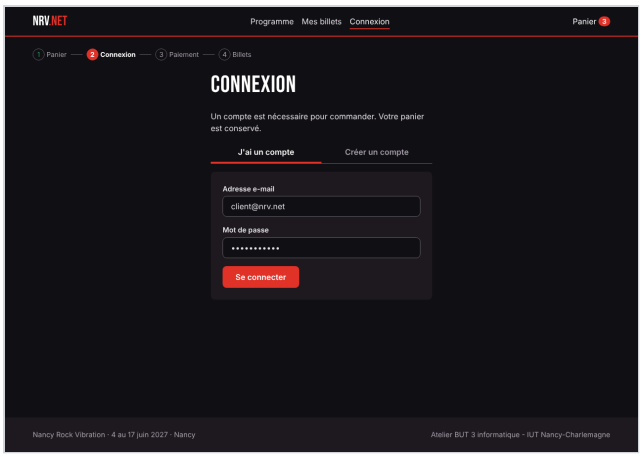


Figure 11 · Connexion (compte obligatoire pour commander)

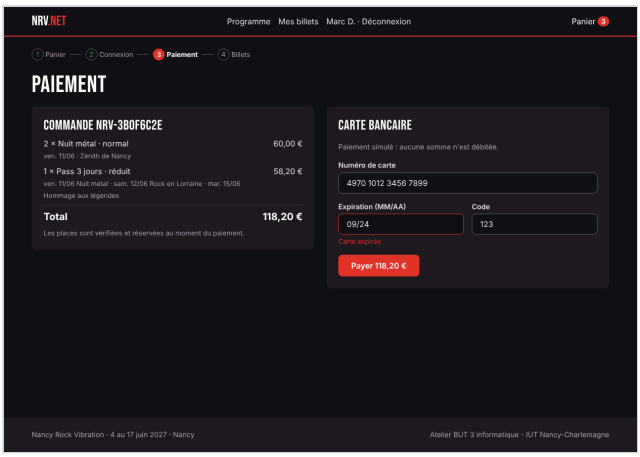


Figure 12 · Paiement : récapitulatif et carte, avec un message d'erreur

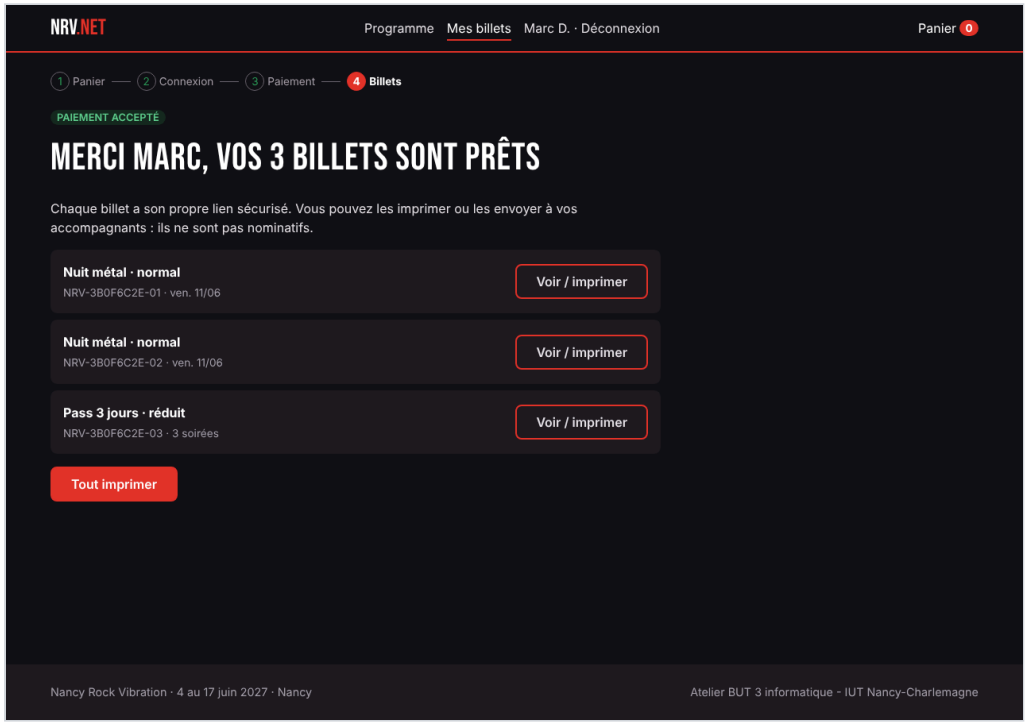


Figure 13 · Confirmation : un lien par billet

4.5 Billets imprimables

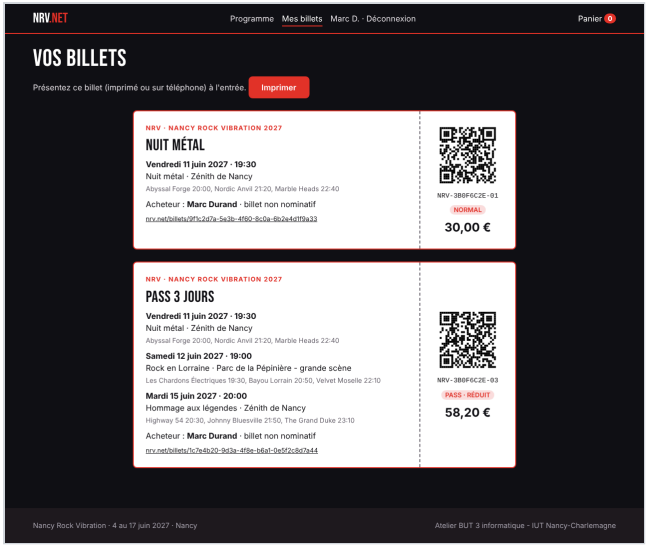


Figure 14 · Billet soirée et Pass 3 jours à l'écran

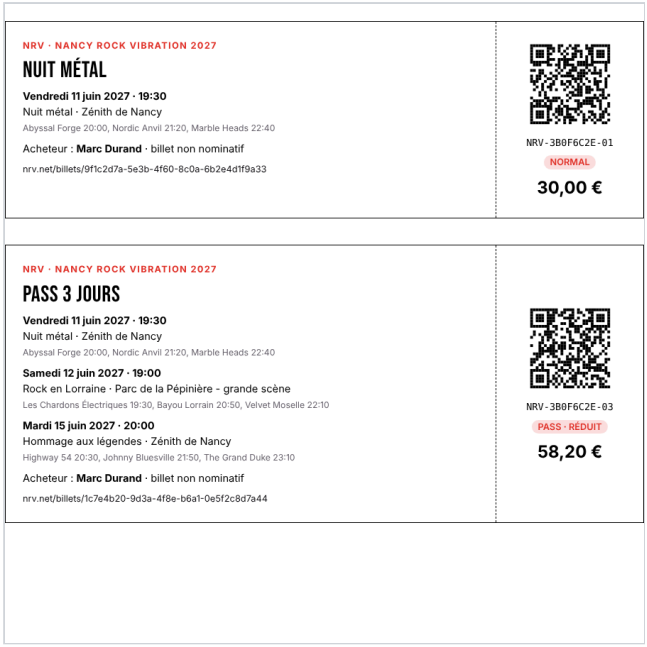


Figure 15 · Rendu à l'impression (`@media print`)

Chaque billet porte le nom de l'acheteur, la référence, la ou les soirées (date, horaire, lieu, spectacles), la catégorie et le prix (F10). À l'impression : en-tête, menu et boutons disparaissent, le billet est en noir et blanc, et il ne se coupe jamais entre deux pages.

4.6 Backoffice : suivi des jauges

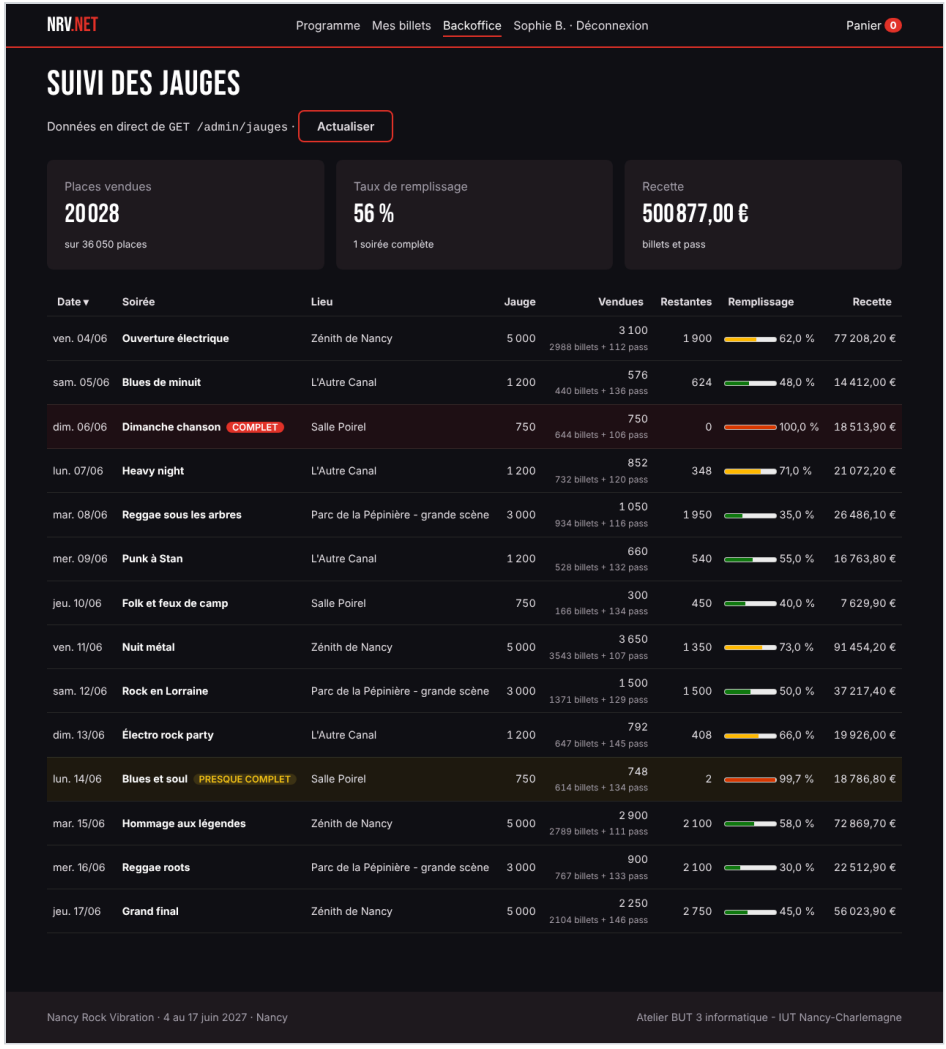


Figure 16 · Jauge et recette de chaque soirée (F11)

Trois indicateurs en haut (places vendues, taux de remplissage, recette), puis une ligne par soirée. Les soirées complètes sont en rouge, celles à plus de 90 % en jaune. Sur téléphone, le tableau défile horizontalement.

Choix de mise en page

- **CSS Grid** pour les zones en deux dimensions : grille des spectacles (`repeat(auto-fill, minmax(16rem, 1fr))` : 1 colonne sur téléphone, 4 sur ordinateur), page soirée (spectacles | achat), paiement (récapitulatif | carte), indicateurs du backoffice.
- **Flexbox** pour les alignements sur une ligne : en-tête et navigation, barre de filtres, lignes du panier, sélecteurs de quantité, étapes de la commande.
- Trois tailles d'écran (mixin Sass `mq()`) : mobile par défaut, tablette dès 600 px, ordinateur dès 1024 px.

5 Environnement technique

Docker Compose

Service	Image	Adresse	Rôle
`db`	mariadb:10.11	127.0.0.1:3307	base de données ; le schéma est joué au premier démarrage
`php`	php:8.3-fpm (Dockerfile)	interne	exécute l'API Slim ; contient Composer
`api`	nginx:1.27	localhost:8080	serveur web de l'API (tout passe par `public/index.php`)
`front`	nginx:1.27	localhost:8081	site statique (`front/public`)
`adminer`	adminer:4	localhost:8082	interface pour consulter la base
`sass`	node:20 (profil dev)	—	recompile le Sass à chaque sauvegarde

Les ports se changent dans le fichier `.env` (utile sur un serveur partagé). Sur Fedora, les volumes ont l'option `:z` pour SELinux.

Installation

```
git clone https://github.com/Kamdinee/Atelier-Projet-de-developpement-web-1.git
cd Atelier-Projet-de-developpement-web-1
git checkout production-kam
cp .env.example .env

docker compose up -d --build
docker compose exec php composer install
docker compose exec php composer seed     # jeu de donnees de test
docker compose exec php composer test     # tests unitaires
```

Jeu de données de test

Le script `api/bin/seed.php` (FakerPHP, graine fixe : les données sont toujours les mêmes) remplit la base avec :

- 4 lieux à Nancy, 8 styles, 14 soirées du 4 au 17 juin 2027, 37 spectacles, 52 artistes ;
- la formule Pass 3 jours (87,30 € / 58,20 €) ;
- 42 utilisateurs, environ 5 600 commandes payées, 18 000 billets soirée et 590 Pass 3 jours, soit environ 500 000 € de recette.

Cas de test remarquables	Id	Situation
Dimanche chanson (6 juin, Salle Poirel)	3	**complète** : tout achat doit être refusé
Blues et soul (14 juin, Salle Poirel)	11	**2 places restantes** : 2 billets passent, 3 sont refusés
Nuit métal (11 juin, Zénith)	8	3 spectacles, grande jauge

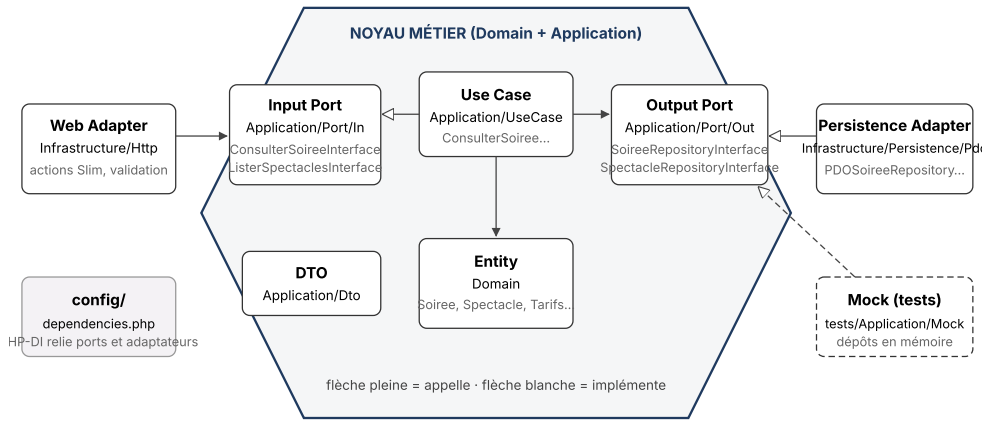
Compte	E-mail	Mot de passe
Administratrice	admin@nrv.net	admin12345
Client	client@nrv.net	client12345
40 clients fictifs	voir la table `utilisateurs`	motdepasse1

6 Backend réalisé (itération 1)

L'itération 1 livre la partie « consultation » de l'API : programme, filtres, pagination, détail des soirées. Le noyau métier est complet pour ces fonctionnalités et couvert par 49 tests unitaires.

6.1 Architecture hexagonale

Le code suit le schéma vu en cours : un **noyau métier** (entités, cas d'usage, ports) qui ne dépend de rien, entouré d'**adaptateurs**. Les ports sont des **interfaces** ; les cas d'usage sont des **services**.



> **Inversion de dépendances**
>
> Le cas d'usage `ConsulterSoiree` demande une soirée à une **interface** (`SoireeRepositoryInterface`) sans savoir qu'il y a PDO derrière. C'est `config/dependencies.php` qui branche l'implémentation PDO (inversion de dépendances). Dans les tests, on branche un faux dépôt en mémoire à la place, sans rien changer au cas d'usage.

Couche	Dossier	Contient · dépend de
Domaine	`src/Domain`	entités (`Soiree`, `Spectacle`, `Lieu`, `Style`), value objects (`Tarifs`, `Image`, `CategorieTarif`, `CriteresSpectacles`…), exceptions métier · **ne dépend de rien** (PHP natif seulement)
Application	`src/Application`	ports d'entrée et de sortie (interfaces), cas d'usage (services), DTO · dépend du domaine seulement
Adaptateur d'entrée	`src/Infrastructure/Http`	actions Slim, validation des paramètres, erreurs JSON, CORS, liens HATEOAS · appelle les ports d'entrée
Adaptateur de sortie	`src/Infrastructure/Persistence/Pdo`	dépôts PDO · implémentent les ports de sortie

6.2 Parcours d'une requête

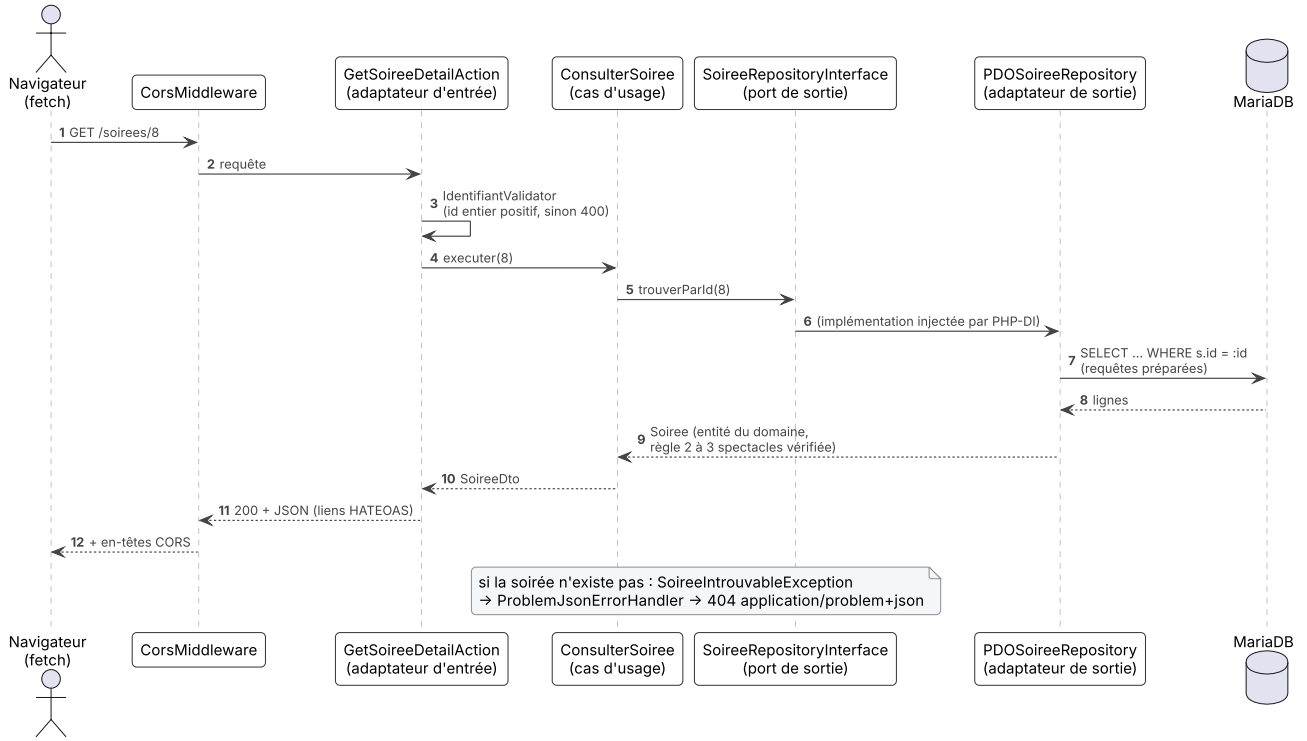


Figure 17 · Ce qui se passe pour `GET /soirees/8`

Les middlewares, dans l'ordre

Slim exécute en premier le middleware ajouté en dernier. Dans `config/middlewares.php` :

1. **CorsMiddleware** : ajoute les en-têtes `Access-Control-*` pour les origines autorisées, répond 204 aux requêtes `OPTIONS`. Il est à l'extérieur pour que même les réponses d'erreur aient les en-têtes CORS.
2. **ErrorMiddleware** avec `ProblemJsonErrorHandler` : transforme toute exception en réponse JSON avec le bon code HTTP.
3. **RoutingMiddleware** : trouve l'action à appeler.
4. **BodyParsingMiddleware** : décode le JSON des requêtes POST (utile à l'itération 2).

6.3 Endpoints disponibles

Route	Paramètres	Action Slim
`GET /`	—	`AccueilAction`
`GET /spectacles`	`date` (AAAA-MM-JJ), `style` (slug), `page` (≥ 1), `size` (1 à 50), tous facultatifs et cumulables	`GetSpectaclesAction`
`GET /spectacles/{id}`	id entier positif	`GetSpectacleAction`
`GET /soirees/{id}`	id entier positif	`GetSoireeDetailAction`
`GET /styles`	—	`GetStylesAction`
`GET /dates`	—	`GetDatesAction`

GET /spectacles?style=metal&size=1


```
{
  "type": "collection",
  "count": 1, "total": 7, "page": 1, "size":
1,
  "filtres": { "date": null, "style": "metal"
},
  "spectacles": [ {
    "spectacle": {
      "id": 9, "titre": "Fer Rouge",
      "date": "2027-06-07", "horaire":
"20:30",
      "image": "https://picsum.photos/…",
      "style": { "slug": "metal",
                 "libelle": "Métal" } },
    "links": {
      "self":   { "href": "/spectacles/9" },
      "soiree": { "href": "/soirees/4" } } }
],
  "links": {
    "self":  { "href": "/spectacles?
style=metal&page=1&size=1" },
    "first": { "href": "…page=1…" },
    "next":  { "href": "…page=2…" },
    "last":  { "href": "…page=7…" } }
}
```


GET /soirees/8 (extrait)


```
{
  "type": "resource",
  "soiree": {
    "id": 8, "nom": "Nuit métal",
    "thematique": "Soirée métal",
    "date": "2027-06-11", "heure_debut":
"19:30",
    "lieu": { "id": 2, "nom": "Zénith de
Nancy",
              "adresse": "rue du Zénith, …" },
    "tarifs": { "normal": 3000, "reduit": 2000
},
    "jauge": 5000, "places_restantes": 1350,
    "complete": false,
    "spectacles": [ {
      "id": 19, "titre": "Abyssal Forge",
      "artistes": [ "Abyssal Forge" ],
      "description": "Riffs massifs…",
      "style": { "slug": "metal", … },
      "horaire": "20:00",
      "images": [ { "url": "…", "alt": "…" }
],
      "video":
"https://www.youtube.com/embed/…"
    }, … ]
  },
  "links": { "self": { "href": "/soirees/8" },
    "spectacles": { "href": "/spectacles?
date=2027-06-11" } }
}
```


Erreurs

```
GET /spectacles?date=2027-02-31&page=0

400 application/problem+json
{
  "type": "https://nrv.net/erreurs/requete-
invalide",
  "title": "Requête invalide",
  "status": 400,
  "errors": {
    "date": "format attendu : AAAA-MM-JJ",
    "page": "entier superieur ou egal a 1
attendu"
  }
}
```

Code	Cas
400	paramètre invalide (date impossible, style mal formé, id non numérique)
404	soirée, spectacle ou route inconnus
405	méthode non autorisée (en-tête `Allow`)
409	places insuffisantes (itération 2)
422	règle métier non respectée
500	erreur interne, sans aucun détail en production

6.4 Sécurité

Risque	Protection
Injection SQL	toutes les requêtes sont préparées (`prepare` + `bindValue`) ; les vraies requêtes préparées du serveur sont forcées (`PDO::ATTR_EMULATE_PREPARES => false`) ; les filtres absents ne sont pas ajoutés au SQL ; une liste d'id (`IN`) est construite avec autant de `?` que de valeurs, toujours liées
Données invalides	validation avant les cas d'usage (`FiltreSpectaclesValidator` , `IdentifiantValidator`) : format de date réel (`checkdate`), slug en `[a-z0-9-]` , bornes de pagination ; puis règles du domaine dans les entités
Fuite d'informations	en production (`APP_ENV=prod`), une erreur 500 ne contient ni message ni trace
Appels depuis un autre site	CORS limité aux origines listées dans `CORS_ORIGINS`
Accès à la base	MariaDB n'est accessible que depuis la machine (`127.0.0.1`)
Dépendances	versions sans faille connue : php-jwt 7 (la version 6 est touchée par la CVE-2025-45769), Slim ≥ 4.15.2, PHPUnit ≥ 11.5.50

Vérifié : `?style=x' OR 1=1 --` est refusé (400) ; passé directement au dépôt, il ne renvoie aucun résultat.

6.5 Tests unitaires

49 tests, 86 assertions, tous verts (PHPUnit 11). Ils testent le noyau métier sans base de données ni Docker.

tests/Domain (entités et value objects)

- soirée de 2 ou 3 spectacles acceptée, 1 ou 4 refusée
- spectacles triés par horaire, spectacle d'une autre soirée refusé
- réservation : la jauge baisse, refus si plus de places que restantes, refus si complète, refus de 0 place
- jauge ≤ capacité du lieu, places vendues ≤ jauge
- tarifs : réduit ≤ normal, pas de négatif
- un seul style par spectacle, artiste obligatoire
- pagination : décalage, nombre de pages

tests/Application (cas d'usage)

- filtres par style, par date, combinés
- pagination, contenu des DTO
- date mal formée refusée
- soirée et spectacle introuvables → exception
- soirée complète signalée
- listes des filtres triées

> **Mocks des ports de sortie**
>
> Les cas d'usage sont testés avec `InMemorySoireeRepository` et `InMemorySpectacleRepository` , qui implémentent les mêmes interfaces que les dépôts PDO.

Lancer les tests : `docker compose exec php composer test` (ou `vendor/bin/phpunit --testdox` pour le détail de chaque test).

Contrôle supplémentaire : en cassant volontairement la règle de jauge dans `Soiree::reserver()` , le test correspondant échoue bien.

7 Bilan et perspectives

État d'avancement

Étape	Réalisé	Restant
1. Conception	scénarios d'usage, diagramme de classes, base de données et schéma SQL, contrat d'API, maquettes de tous les écrans (ordinateur, téléphone, impression du billet)	—
2. Environnement	dépôt Git, Docker Compose, script SQL, jeu de données de test, structure Sass, squelette de l'architecture hexagonale, README	—
3. Production	backend de consultation : entités et ports, adaptateurs PDO, actions Slim et routes, conteneur de dépendances ; 49 tests unitaires	intégration front des écrans Programme et Détail soirée

Itération 2

- Authentification : inscription, connexion, mots de passe hachés (bcrypt), jeton JWT, middlewares d'authentification et de rôle administrateur.
- Commande : `POST /commandes` avec middleware de validation ; prix recalculés par le serveur ; Pass 3 jours sur 3 soirées distinctes.
- Paiement : transaction avec verrouillage des soirées (`SELECT … FOR UPDATE`), réservation par `Soiree::reserver()`, création des billets et de leur jeton UUID, annulation complète en cas de refus.
- Billets et backoffice : `GET /billets/{token}`, `GET /commandes/{id}/billets`, `GET /admin/jauges`.
- Front : branchement des écrans sur l'API avec `fetch` et Handlebars, à partir des maquettes.
- Tests unitaires du domaine pour le Pass 3 jours, la commande et le paiement.

Déploiement

Le déploiement sur **docketu.iutnc.univ-lorraine.fr** est préparé : les ports, le mode d'exécution (`APP_ENV=prod`), le secret JWT et les origines autorisées (CORS) sont paramétrés dans le fichier `.env`, et le front détermine l'adresse de l'API à partir de celle du serveur. La procédure est décrite dans le README.